Distance
- unit: Unit
- meters: double

Distance(Unit unit, double meters)
+ of(Unit unit, double value): Distance
+ as(Unit unit): double

These are very commonly used data types; connecting these to everything else would be very messy and doesn't provide much value.

<<interface>> Location

+ coordinates(): Coordinates
+ estimateState(): Optional<State>

+ <<enumerated>> Distance.Unit

METERS(new Ratio(1, 1)),
KILOMETERS(new Ratio(1000, 1)),
FEET(new Ratio(381, 1250)),
MILES(new Ratio(50, 80467)),

+ fromMeters(double meters): double
+ toMeters(double value): double

Address
- streetAddress: String
- city: String
- state: State
- zipCode: Optional<ZipCode>

+ Address(String streetAddress, String city, String state, String country)

Coordinates
- latitude: double
- longitude: double

+ Coordinates(double latitude, double longitude)

Ratio
- numerator: int
- denominator: int

+ decimal(): double

ZipCode
- value: String

+ ZipCode(String value)

<<enumerated>> State

SC,
NC,
GA,
...

+ fromAbbreviation(String abbreviation): Optional<State>

DrivingRoute
- start: Location
- end: Location
- points: List<Coordinates>

+ Coordinates(double latitude, double longitude)

DataConstants

...

DataReader

+ readDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun): D
+ readDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun, File file): D
+ readDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun, Reader reader): D

DataWriter

+ writeDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun)
+ writeDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun, File file)
+ writeDatabase(JsonType<BaseDatabase<V>> type, Function<BaseDatabase<V>, D> fun, Reader reader)

HurricaneUI

...

...

HurricaneApplication

+ RANDOM: Random
- INSTANCE: HurricaneApplication
- loggedInUser: Optional<User>
- loggedInUserKey: Optional<Key<User>>

- userDb: UserDb
- requestDb: RequestDb
- shelterDb: ShelterDb
- hurricaneDb: HurricaneDb

- HurricaneApplication()
- HurricaneApplication(UserDb userDb, RequestDb requestDb, ShelterDb shelterDb, HurricaneDb hurricaneDb)
+ getInstance(): HurricaneApplication
+ mockApplication(): HurricaneApplication

+ createAccount(String username, String password): Key<User> throws AccountCreationError
+ login(String username, String password): User throws AccountLoginError
+ logout()

+ addMember(Being being)
+ editUser(User user, Consumer<User> consumer) throws PermissionsError
+ getUser(Key<User> key): User
+ searchUser(UserQuery query): Set<Key<User>>

+ createRequest(Request request): Key<Request>
+ editRequest(Request request, Consumer<Request> consumer) throws PermissionsError
+ getRequest(Key<Request> key): Request
+ searchRequest(RequestQuery query): Set<Key<Request>>
+ closeRequest(Key<Request> key, CloseReason reason) throws PermissionsError
+ claimProxyRequest(Key<Request> request)
+ addParticipantToRequest(Key<Request> request)

+ createShelter(Shelter shelter): Key<Shelter>
+ editShelter(Shelter shelter, Consumer<Shelter> consumer) throws PermissionsError
+ getShelter(Key<Shelter> key): Shelter
+ searchShelter(ShelterQuery query): Set<Key<Shelter>>
+ removeShelter(Key<Shelter> key) throws PermissionsError

+ getHurricane(Key<Hurricane> key): Hurricane
+ searchHurricane(HurricaneQuery query): Set<Key<Hurricane>>

Key<V>
- uuid: UUID

+ Key(UUID uuid)
+ random(): Key<V>
+ fromString(String name): Key<V>
+ jsonKeyString(): String

<<interface>> Searchable<V, Q>

+ search(Q query): Set<Key<V>>

Database<V>

+ base: BaseDatabase<V>

~ Database()
~ Database(BaseDatabase<V> base)
+ create(V element): Key<V>
+ get(Key<V> key): V
+ remove(Key<V> key): V
+ edit(Key<V> key, Consumer<V> consumer)
+ search(IQuery<V> query): Set<Key<V>>

<V -> V>
<Q -> IQuery<V>>

<<interface>> IQuery<V>

+ matches(V obj): boolean

BaseDatabase<V>

entries: Map<Key<V>, V>

+ BaseDatabase(Map<Key<V>, V> entries)
+ create(V element): Key<V>
+ get(Key<V> key): V
+ remove(Key<V> key): V
+ edit(Key<V> key, Consumer<V> consumer)
~ getEntries(): Map<Key<V>, V>

+ BaseDatabase.BaseDatabaseJsonAdapter<V>

- valueAdapter: JsonAdapter<V>
+ FACTORY: AdapterFactory

+ BaseDatabaseJsonAdapter(Jsonb jsonb, Type generic)
+ toJson(JsonWriter writer, BaseDatabase<V> db)
+ fromJson(JsonReader reader): BaseDatabase<V>

You may see builder() and @Query classes in many of the Database items. These will be autogenerated, and so are not included in the UML.

<V -> Hurricane>
<V -> Shelter>
<V -> Request>
<V -> User>

HurricaneDb

- instance: HurricaneDb

- HurricaneDb()
- HurricaneDb(BaseDatabase<Shelter> base)
+ getInstance(): HurricaneDb

ShelterDb

- instance: ShelterDb

- ShelterDb()
- ShelterDb(BaseDatabase<Shelter> base)
+ getInstance(): ShelterDb

RequestDb

- instance: RequestDb

- RequestDb()
- RequestDb(BaseDatabase<Request> base)
+ getInstance(): RequestDb

UserDb

- instance: UserDb

- UserDb()
- UserDb(BaseDatabase<User> base)
+ getInstance(): UserDb

@Query Hurricane
- name: String
- formed: LocalDate
- dissipated: Optional<LocalDate>
- affectedAreas: Set<State>

+ Hurricane(String name, LocalDate formed, ...)
+ isAffected(Location location): boolean

@Query Shelter
- owner: Key<User>
- name: String
- location: Location
- resources: Set<Resource>

+ Shelter(User owner, String name, ...)
+ builder(): Shelter.Builder

@Query Request
- author: Key<User>
- owner: Optional<Key<User>>
- proxy: Boolean
- participants: Set<Key<User>>
- date: LocalDate
- closeReason: Optional<CloseReason>
- location: Location
- description: String
- beings: Set<Being>
- requiredResources: Set<Resource>
- requiredSpecialties: Set<Specialty>

+ Request(User author, LocalDate date, ...)
+ canEdit(User user): boolean
+ addParticipant(User user)
+ claimProxy(User user)
+ close(CloseReason reason)
+ isClosed(): boolean
+ builder(): Request.Builder

<<interface>> Specialty

Transportation
- capacity: int
- allowedPets: Set<Species>

+ Transportation(int capacity, Set<Species> allowedPets)

Unspecified
- value: String

+ Unspecified(String value)

Authentication.AuthKey
- salt: byte[]
- hash: byte[]

+ AuthenticationInfo(byte[] salt, byte[] hash)
+ fromPassword(String password): Authentication.AuthKey
+ checkPassword(String password): boolean

Authentication.Username

- Username()
+ isValid(String username): boolean

Authentication

- Authentication()

Authentication.Password

- Password()
+ isValid(String password): boolean

The Resource interface contains and refers to multiple internal classes that will only complicate the diagram: AllowedPets, AnimalCapacity, HumanCapacity, MedicalStaff, WheelchairAccessible, and Custom

<<interface>> Resource

Species
- value: String

+ Species(String value)

SpecialNeed
- name: String
- description: String

+ SpecialNeed(String name, String description)

Being
firstName: String
lastName: String
birthDate: LocalDate
specialNeeds: List<SpecialNeed>

+ Being(String firstName, String lastName, LocalDate birthDate, List<SpecialNeed> specialNeeds)

CloseReason
- type: CloseReason.Type
- message: String

- CloseReason(CloseReason.Type type, String message)
+ resolved(String message): CloseReason
+ retracted(String message): CloseReason

+ <<enumerated>> CloseReason.Type

RESOLVED
RETRACTED

Animal
- species: Species

+ Animal(String firstName, String lastName, LocalDate birthDate, List<SpecialNeed> specialNeeds, Species species)

Person
- phoneNumber: PhoneNumber

+ Person(String firstName, String lastName, LocalDate birthDate, List<SpecialNeed> specialNeeds, PhoneNumber phoneNumber)

PhoneNumber
- countryCode: String
- areaCode: String
- localNumber: String

+ PhoneNumber(String countryCode, String areaCode, String localNumber)

<<enumerated>> Role

CITIZEN("Citizen")
EMS("Emergency Services")

Role(string: String)
+ toString(): String

@Query User
- username: String
- login: AuthKey
- person: Person
- role: Role
- location: Location
- family: Set<Being>
- ownedRequests: Set<Key<Request>>
- participatingRequests: Set<Key<Request>>
- ownedShelters: Set<Key<Shelter>>

+ User(String username, AuthKey login, Person person, Role role, ...)
+ addMember(Being being)